

MLA03 – REINFORCEMENT LEARNING ASSIGNMENT

Assignment Information

Department:	(AI & ML)
Programme:	FACULTY OF ENGINEERING
Course Code & Course Name	MLA03 & REINFORCEMENT LEARNING
Academic Year / Batch	2026-2027
Faculty Name	Dr.G. A. SENTHIL
Assignment Title	RL-Based Patient Health Monitoring and Treatment Recommendation System Using Markov Decision Process and Bellman Equations
Date of Issue	01-09-2026
Date of Submission	01-09-2026
Maximum Marks	100
Course Outcome(s) - CO	CO1-Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems. CO2 - Apply Dynamic Programming methods to solve reinforcement learning problems and derive optimal policies. CO3 - Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. CO4-Evaluate model-based reinforcement learning approaches for prediction, planning, and policy optimization. CO5-Apply hierarchical, meta-learning, and multi-agent RL approaches to solve complex decision-making problems.
Bloom's Taxonomy Level	BL3 (Apply) - <i>Dynamic Programming methods to solve reinforcement learning problems and derive optimal policies for decision-making tasks.</i> BL4 (Analyse) - <i>Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments.</i> BL5 (Evaluate) - <i>Model-based</i>

	<i>reinforcement learning approaches for prediction, planning, and policy optimization. BL6 (Create) - Develop an MDP-based solution for an intelligent traffic signal system by defining the state space, action space, reward function, transition model, and optimal policy.</i>
SDG Mapping (SDG 1-17)	SDG 3 – Good Health and Well-being: The primary SDG for this project, focusing on continuous patient health monitoring, personalized treatment recommendations, improved treatment effectiveness, recovery rates, and overall patient well-being. SDG 9 – Industry, Innovation and Infrastructure: The project uses Reinforcement Learning, Markov Decision Processes, and Bellman equations to develop an innovative AI-based healthcare decision-support system. SDG 10 – Reduced Inequalities: The system can support accessible and consistent healthcare recommendations, particularly through remote patient monitoring and intelligent decision support. SDG 12 – Responsible Consumption and Production: Optimal treatment recommendations can help reduce unnecessary interventions and promote efficient utilization of healthcare resources. SDG 17 – Partnerships for the Goals: The system can encourage collaboration among healthcare providers, researchers, AI developers, pharmaceutical organizations, and public-health institutions.
Industry / Societal Relevance	This project is relevant to the healthcare, hospitals, digital health, pharmaceutical, medical AI, and remote patient monitoring industries. It provides an intelligent framework for monitoring changing patient conditions and recommending appropriate treatment strategies using Reinforcement Learning. From a societal perspective, it can support personalized healthcare, early identification of health deterioration, improved treatment effectiveness and recovery, reduced unnecessary medical interventions, efficient healthcare resource utilization, and better access to healthcare through remote monitoring. Efficient water utilization, and sustainable agricultural practices.

Question:

RL-Based Patient Health Monitoring and Treatment Recommendation System Using Markov Decision Process and Bellman Equations

A healthcare organization wants to develop an intelligent Reinforcement Learning-based patient health monitoring and treatment recommendation system that continuously observes patient health parameters, identifies changing health conditions, evaluates treatment outcomes, and recommends appropriate treatment strategies. The system uses Markov Decision Process (MDP) and Bellman equations to model sequential healthcare decisions and learn an optimal treatment policy.

The system considers patient health conditions as states, treatment interventions as actions, clinical outcomes as rewards, and changing patient conditions as state transitions. The objective is to maximize long-term patient outcomes while improving treatment effectiveness and recovery.

The overall assignment can address the following:

1. Formulate patient health monitoring and treatment recommendation as a Markov Decision Process (MDP) by defining states, actions, transition dynamics, rewards, and policies.
2. Design a Dynamic Programming-based solution using Value Iteration or Policy Iteration to determine an optimal treatment policy under different patient health conditions.
3. Implement and compare Monte Carlo, SARSA, and Q-Learning approaches for sequential treatment recommendation.
4. Develop a Deep Q-Network (DQN)-based healthcare agent and investigate advanced approaches such as Double DQN, Dueling DQN, and Prioritized Experience Replay.
5. Develop a policy-based or Actor-Critic solution such as REINFORCE, A2C, PPO, or DDPG for treatment decision-making.
6. Construct a model-based Reinforcement Learning framework that learns patient-state transition dynamics and predicts possible future health outcomes.
7. Develop an optional Hierarchical or Multi-Agent RL architecture where different agents monitor patient health, treatment effectiveness, medication response, and risk conditions.
8. Integrate the components into a complete healthcare architecture:

Patient Data → Preprocessing → Health-State Identification → MDP → RL Agent → Treatment Recommendation →

Clinical Outcome → Reward → Policy Improvement

9. Evaluate the learned policy using treatment effectiveness, recovery rate, cumulative reward, convergence rate, and decision performance.
10. Analyse the ethical, clinical, economic, privacy, safety, and societal implications of AI-based treatment recommendation.

Assignment Problem / Challenge

Modern healthcare systems need to continuously monitor patient conditions and select appropriate treatment strategies as patient health changes over time. Conventional rule-based treatment systems generally rely on predefined thresholds or static clinical guidelines and may not adequately adapt to sequential changes in patient conditions.

The assignment challenges students to design and develop an RL-based patient health monitoring and treatment recommendation system that observes patient health conditions, models them as states, selects treatment interventions as actions, receives clinical outcomes as rewards, and continuously improves its treatment policy.

The proposed system must balance treatment effectiveness, patient recovery, safety, and long-term clinical outcomes while ensuring that AI recommendations remain within appropriate healthcare constraints. This framing follows the same MDP/RL structure used in the uploaded assignment, adapted from agricultural resource decisions to sequential healthcare decisions.

Problem Statement Design and Development of an RL-Based Patient Health Monitoring and Treatment Recommendation System Using Markov Decision Process and Bellman Equations

Modern healthcare faces challenges in continuously monitoring changing patient conditions and selecting appropriate treatment strategies based on individual patient responses. Traditional rule-based and static treatment recommendation systems may not adequately adapt to changes in vital signs, symptoms, disease severity, treatment response, and recovery status.

Therefore, there is a need to design and develop an **intelligent Reinforcement Learning-based healthcare system** that continuously observes patient health conditions, represents them as states in a Markov Decision Process, selects suitable treatment interventions as actions, and evaluates clinical outcomes using a reward function. The system should use **Bellman equations and reinforcement learning algorithms** to learn an optimal treatment policy that adapts to changing patient conditions.

The proposed system should maximize **treatment effectiveness and patient recovery** while minimizing undesirable outcomes, unnecessary interventions, and treatment-related risks.

Requirements and Constraints Requirements

- Collect real or simulated patient health data.
- Include relevant parameters such as vital signs, symptoms, disease severity, treatment history, and patient response.

- Define appropriate **states, actions, rewards, transition probabilities, and policies**.
- Formulate the healthcare problem as an MDP.
- Implement Bellman-based value optimization.
- Implement suitable RL algorithms such as **Q-Learning, SARSA, DQN, PPO, or Actor-Critic**.
- Develop the system using Python and appropriate libraries.
- Generate treatment recommendations based on changing patient states.
- Evaluate treatment effectiveness and recovery rate.
- Calculate cumulative reward and convergence rate.
- Compare at least two RL algorithms.
- Present results using graphs, tables, learning curves, and policy comparisons.
- Include safety, privacy, ethical, and human-supervision considerations.

Constraints

- Real-world clinical datasets may have limited availability.
- Patient health conditions can be highly variable and uncertain.
- Medical data may contain missing, noisy, or incomplete observations.
- Incorrect treatment recommendations can have serious consequences.
- RL exploration must not result in unsafe treatment decisions.
- The quality of the learned policy depends heavily on the reward-function design.
- A simulated environment may not completely represent real clinical conditions.
- Patient data must be handled with appropriate privacy and security measures.
- Real-world deployment requires validation and supervision by qualified healthcare professionals.

1. Problem Understanding and Formulation What is the problem?

An intelligent healthcare decision-making system must be developed that can:(a) collect and interpret patient health observations,(b) identify the patient's current health condition,(c) formulate healthcare treatment as a Markov Decision Process,(d) learn optimal treatment strategies using Reinforcement Learning,(e) adapt treatment recommendations according to changing patient conditions,(f) maximize treatment effectiveness and recovery, and(g) provide an interpretable decision-support system for healthcare professionals.

Expected outcomes

The expected outcomes are:

- A clearly defined **MDP formulation** containing states, actions, transition dynamics, reward function, and policy.
- A trained **RL agent** capable of selecting treatment actions.
- A comparison of at least **two RL algorithms**.
- An optimal or near-optimal treatment policy.
- Improved treatment effectiveness and recovery rate.
- Quantitative results for **cumulative reward and convergence**.
- A patient monitoring and recommendation dashboard.
- A complete healthcare decision-making architecture.
- Analysis of safety, ethics, privacy, and human oversight.

Available Information/Data

The system can use a real or synthetic healthcare dataset containing:

- Patient age
- Gender, where appropriate and ethically justified
- Vital signs
- Heart rate
- Blood pressure
- Temperature
- Oxygen saturation
- Blood glucose
- Symptoms
- Disease severity
- Medication/treatment history
- Treatment response
- Recovery status
- Follow-up observations

For an academic implementation, a **synthetic patient environment** can also be created when real longitudinal treatment data is unavailable.

Assumptions

- Patient observations provide sufficient information to estimate the current health state.
- Patient health conditions change over time.
- Available treatment interventions are predefined.
- Each treatment action can produce different clinical outcomes.
- The reward function represents treatment effectiveness and patient recovery appropriately.
- Patient-state transitions can be estimated from historical or simulated data.
- The RL agent learns from repeated interactions with the healthcare environment.
- Real-world treatment decisions remain under qualified healthcare-professional supervision.

2. Solution / Design / Methodology

2.1 Patient Health- State Classification

Patient health observations are first processed to identify the patient's current health state. The target variable is **Crop Water Stress Risk**.

Heart Abnormal	Rate	Blood Abnormal	Pressure	Oxygen Saturation Low	Patient Health Risk
Yes		Yes		Yes	High
No		No		No	Low
Yes		Yes		No	High
No		Yes		No	Low

For the four illustrative records, there are two High-risk and two Low-risk samples. Therefore,

$$\text{Entropy}(S) = -p_H \log_2 p_H - p_L \log_2 p_L$$

$$\text{Entropy}(S) = -(0.5 \log_2 0.5) - (0.5 \log_2 0.5) = 1 \text{ bit}$$

For a candidate feature AA, Information Gain is:

$$IG(S,A)=Entropy(S)-\sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

Splitting on **Heart Rate < 30%** produces two pure groups in this illustrative dataset. Hence, and

$$Entropy(S|Heart\ Rate)=0 \quad IG(S,SoilMoisture)=1-0=1 \text{ bit}$$

Thus, heart-rate abnormality is identified as the most informative feature in this simple example.

Blood pressure, oxygen saturation, temperature, symptom severity, and other health parameters can subsequently be ranked using the same criterion.

The feature-ranking stage helps the healthcare system prioritize important patient parameters before the RL agent makes treatment recommendations.

2.2 Supervised Model Comparison

Before reinforcement learning is applied, supervised learning models are used to estimate patient health risk, disease severity, or treatment response.

Model	Illustrate Validation Accuracy	Notes
Gaussian Naive Bayes	0.82	Fast and lightweight; assumes feature independence
Logistic Regression	0.86	Interpretable probability-based prediction
Decision Tree	0.84	Easy-to-understand irrigation decision rules
Random Forest	0.91	Strong nonlinear prediction and robust to feature interactions

The Random Forest model can be selected as the primary patient-risk classifier because of its illustrative higher validation accuracy and ability to model nonlinear relationships among patient health parameters. A regression model can additionally predict a continuous health outcome:

$$\hat{W} = f(X) \text{ where } W = f(X)$$

where X represents patient health features and Y represents the predicted health outcome or recovery score.

The supervised prediction layer provides the RL agent with an informed estimate of the patient's current health condition.

2.3 Clustering and Dimensionality Reduction

K-means clustering with $K=3$ is applied to standardized agricultural features such as soil moisture, temperature, humidity, rainfall, and crop growth indicators.

Agricultural Cluster	Centroid Profile	Illustrative Size
Cluster 1	Stable vital signs, low symptom severity	50%
Cluster 2	Moderate abnormalities, moderate symptoms	32%
Cluster 3	Severe abnormalities, high symptom severity	18%

The third cluster represents high-risk patient conditions and can be used to evaluate whether the RL agent appropriately selects more intensive monitoring or treatment actions.

The K-means objective is:

$$J = \sum_{i=1}^n \|x_i - \mu_{c_i}\|^2$$

where x_i is a patient observation and μ_{c_i} is the centroid of its assigned cluster. PCA can then reduce the dimensionality: $Z = XP$ where P contains the principal component directions.

For an illustrative experiment, the first two principal components can be used to visualize patient health states in a compact two-dimensional representation.

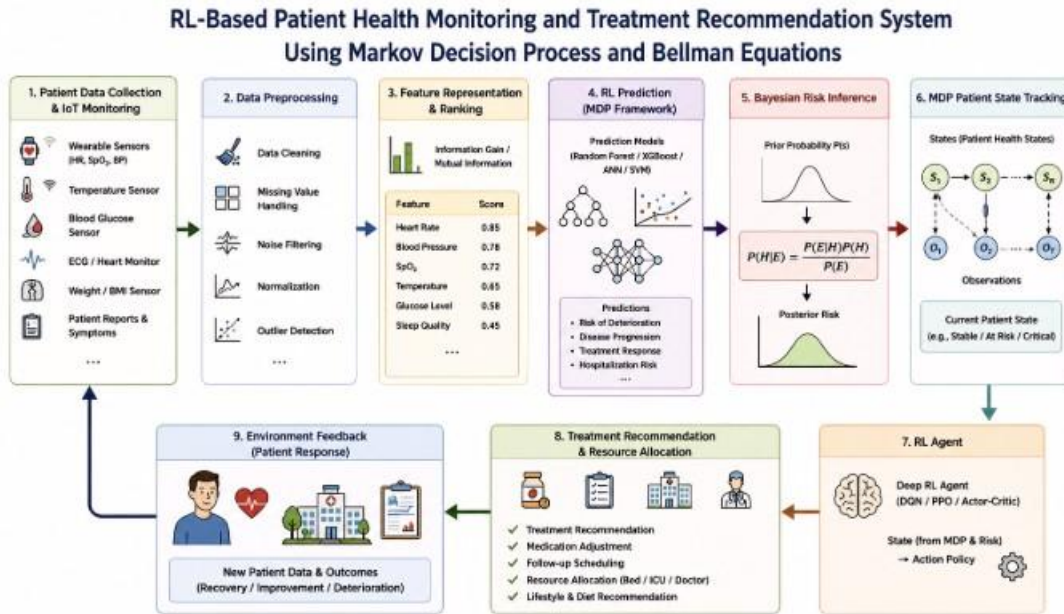
2.4 Bayesian Network for Agricultural Risk Inference

A Bayesian network can be developed to model relationships between patient health factors, disease severity, and treatment response.

The conceptual structure is:

Vital Signs → Disease Severity → Patient Risk → Treatment

Response Illustrative probabilities can be:



Illustrative conditional probabilities are:

Node / Condition	Probability
High Heart Rate	0.35
Abnormal Blood Pressure	0.30
Health Risk High Heart Rate = True	0.65
Health Risk High Heart Rate = False	0.15
Health Risk Abnormal BP = True	0.70
Health Risk Abnormal BP = False	0.20

The Bayesian network estimates:

$P(\text{PatientRisk}|\text{Evidence})$

Using evidence such as high heart rate and abnormal blood pressure, the posterior probability of high patient risk increases compared with the prior probability.

This component provides an interpretable probabilistic explanation before the RL agent selects an action.

2.5 Hidden Markov Model for Crop-State Tracking

Patient health conditions can change continuously over time. Therefore, an HMM can be used to represent hidden health states:

$$S=\{\text{Stable,Mild,Moderate,Severe,Recovered}\}S=\{\text{Stable,Mild,Moderate,Severe,Recovered}\}$$

The transition matrix is:

From / To	Stable	Mild	Moderate	Severe	Recovered
Stable	0.85	0.15	0.04	0.01	0.00
Mild	0.15	0.65	0.15	0.05	0.00
Moderate	0.05	0.15	0.60	0.15	0.05
Severe	0.01	0.04	0.15	0.70	0.10
Recovered	0.05	0.02	0.01	0.00	0.92

The observation space contains:

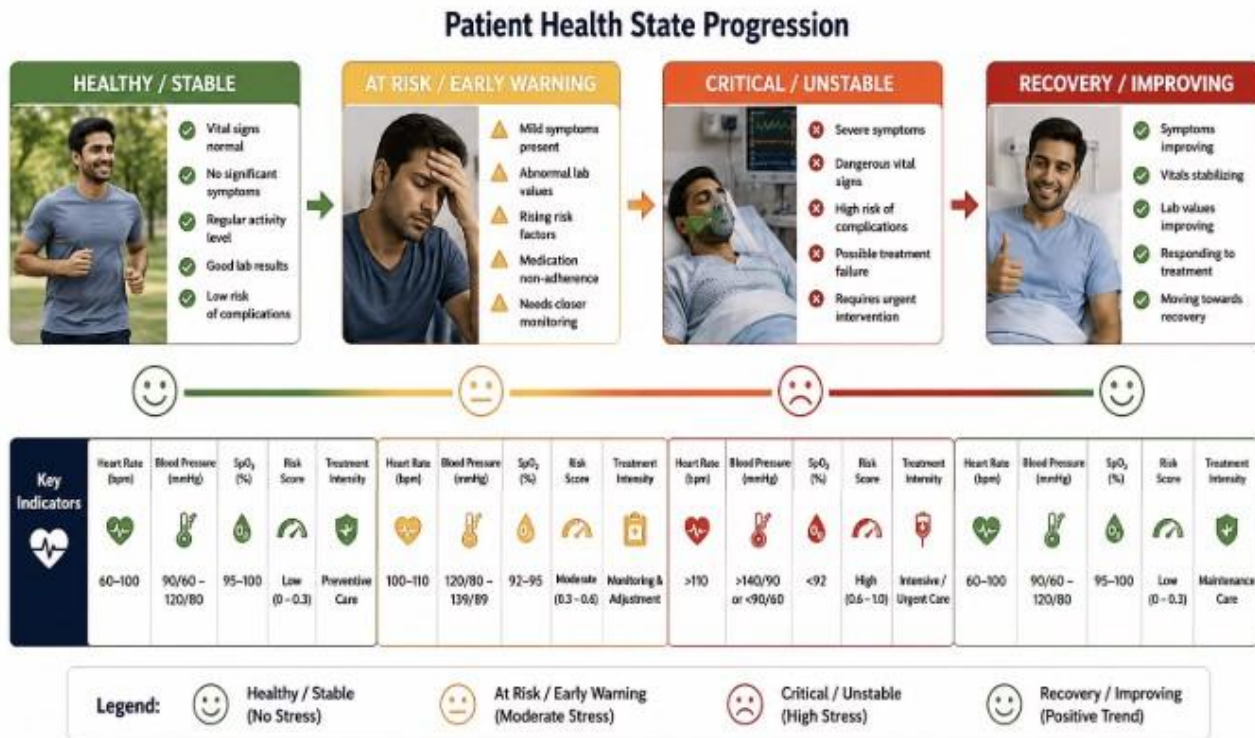
$$O = \{ \text{Normal, Abnormal} \}$$

For an observation sequence:

Normal → Abnormal → Abnormal

the Viterbi algorithm can estimate the most probable hidden state sequence as:

Stable → Moderate → Severe → Improving



This allows the system to identify gradual health deterioration rather than depending only on a single abnormal measurement.

2.6 Reinforcement Learning for Autonomous Resource Allocation

The central component is an autonomous Reinforcement Learning agent that determines appropriate treatment-support actions.

The healthcare environment is modeled as a Markov Decision Process:

$$MDP=(S,A,P,R,\gamma)$$

State Space

The state sts_t contains:

$$=[H_t, B_t, O_t, T_t, G_t, D_t,$$

$R_t]$ where:

- H_t = heart rate
- B_t = blood pressure
- O_t = oxygen saturation
- T_t = temperature

- GtG_t = glucose level
- DtD_t = disease/symptom severity
- RtR_t = previous treatment response

Action Space

$A = \{a_0, a_1, a_2, a_3\}$ where:

- a_0 : Continue current treatment/monitoring
- a_1 : Treatment adjustment
- a_2 : Change treatment strategy
- a_3 : Escalate monitoring/clinical review

These actions should be treated as **decision-support actions**, not autonomous medical prescriptions.

Reward Function

The reward should balance health improvement, recovery, safety, and unnecessary intervention.

A possible reward formulation is:

$R_t = \alpha C_t - \beta W_t - \gamma E_t - \delta S_t$ where:

- H_t = health improvement
- Rec_t = recovery benefit
- $Risk_t$ = deterioration penalty
- I_t = unnecessary intervention penalty
- $\alpha, \beta, \gamma, \delta$ = weighting parameters

A possible reward structure is:

Condition	Reward
Patient recovered	+15
Significant improvement	+10

Mild improvement	+5
Stable condition	+2
No improvement	0
Condition worsens	-10
Severe deterioration	-15
Unsafe action	-20

2.7 Autonomous Multi-Agent Extension

For a large healthcare monitoring system, multiple RL agents can operate cooperatively.

Example agents include:

1. **Patient Monitoring Agent** – monitors vital signs and symptoms.
2. **Risk Assessment Agent** – evaluates patient deterioration risk.
3. **Treatment Agent** – recommends treatment-support actions.
4. **Recovery Agent** – tracks recovery progress.
5. **Resource Agent** – considers healthcare-resource requirements.

The global optimization objective can be represented as:

$$\max_{\pi_1, \dots, \pi_N} E \left[\sum_{t=0}^T \gamma^t R_t^{\text{global}} \right] \quad \text{max}_{\pi_1, \dots, \pi_N}$$

This enables the system to optimize patient outcomes rather than focusing only on an individual treatment decision..

2.8 Integrated Autonomous Agriculture Architecture

Patient Data → Data Preprocessing → Feature Ranking → ML Prediction → Bayesian Risk Inference → HMM Patient-State Tracking → RL Agent → Treatment Recommendation → Patient Outcome

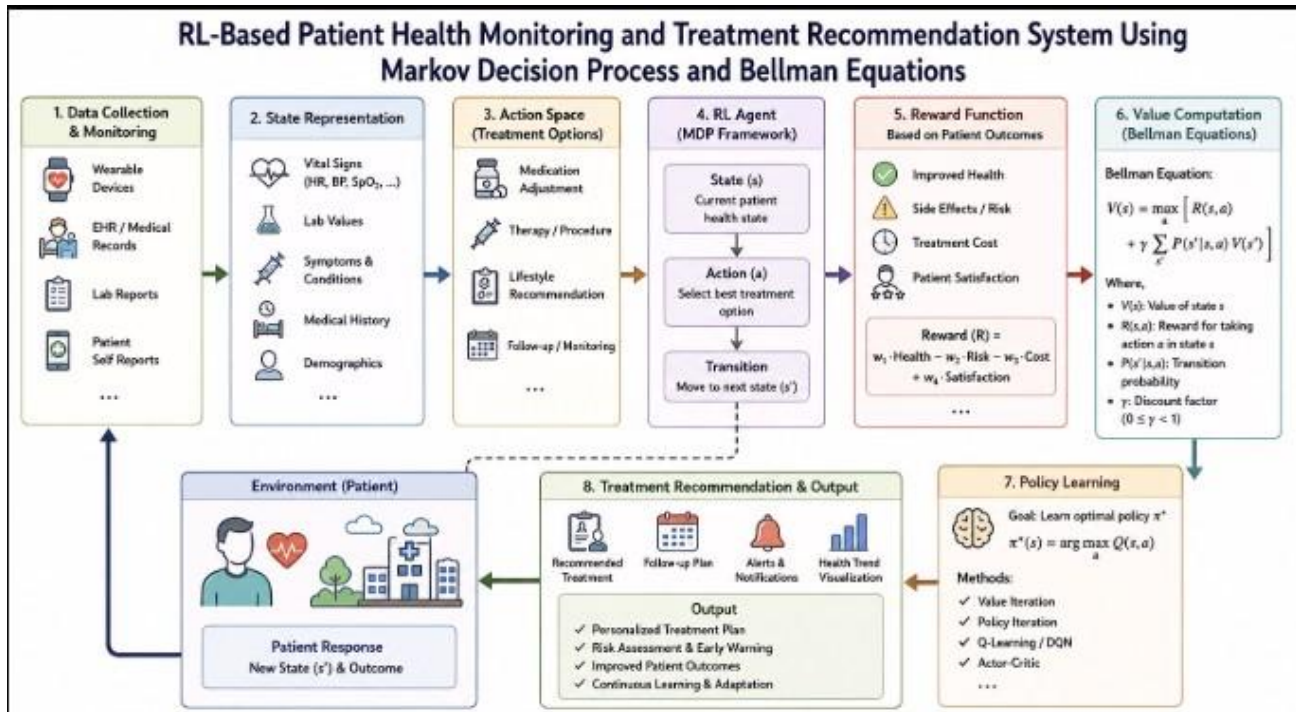
The architecture operates as a continuous closed-loop system:

Observation_t → State_t → Action_t → Reward_t → State_{t+1}

Observation_t → State_t → Action_t → Reward_t → State_{t+1}

The RL agent continuously receives feedback from patient observations and treatment outcomes. Therefore, it learns an adaptive treatment policy rather than a fixed decision rule.

The architecture operates as a continuous closed-loop system:



Observation_t→State_t→Action_t→Reward_t→State_{t+1}Observation_t \rightarrow State_t
\rightarrow Action_t \rightarrow Reward_t \rightarrow State_{t+1}

The RL agent continuously receives feedback from soil sensors, weather conditions, crop status, and water availability. It therefore learns an adaptive policy rather than following a fixed irrigation schedule.

3. Use of Modern Tools

The proposed system can be implemented using the following tools:

Tool / Technology	Purpose
Python	Overall implementation of the patient health monitoring and RL system
NumPy	Numerical calculations and Q-table operations
Pandas	Patient health data processing and analysis

Matplotlib	Visualization of rewards, learning curves, and patient-state transitions
Scikit-learn	Data preprocessing and basic patient-health data analysis
Gymnasium	Creating and simulating the patient health environment
Streamlit	Developing the patient health monitoring and treatment recommendation dashboard
CSV / Healthcare Dataset	Providing patient health observations and treatment-outcome data

4. Representative implementation:

```
# =====
# PATIENT HEALTH MONITORING AND TREATMENT RECOMMENDATION SYSTEM
# Reinforcement Learning - Multi-Armed Bandit
# =====

import numpy as np
import pandas as pd
import matplotlib.pyplot
as plt

# Reproducibility
np.random.seed(42)

# =====
# 1. PATIENT STATES
# =====

states = [
    "Stable",
    "Moderate Risk",
    "Severe Risk"
]
```



```
# =====
```

```
# 2. TREATMENT ACTIONS
```

```
# =====
```

```
actions = [  
    "Routine Monitoring",  
    "Basic Treatment",  
    "Intensive Treatment",  
    "Clinical Intervention"  
]
```

```
# =====
```

```
# 3. SIMULATED REWARD PROBABILITIES
```

```
# =====
```

```
# Probability of successful patient outcome.
```

```
# These are simulated values for academic demonstration.
```

```
reward_probabilities = {
```

```
    "Stable": [  
        0.80, # Routine Monitoring  
        0.60, # Basic Treatment  
        0.40, # Intensive Treatment  
        0.20 # Clinical Intervention  
    ],
```

```
    "Moderate Risk": [  
        0.40,  
        0.75,  
        0.85,
```

```

        0.60
    ],

    "Severe Risk": [
        0.20,
    0.45,
        0.80,
        0.90
    ]
}

# Display reward probability table

print("\n=====")
print("SIMULATED TREATMENT SUCCESS PROBABILITIES")
print("=====\\n")

probability_table = pd.DataFrame(
    reward_probabilities,
    index=actions
)

print(probability_table)

# =====
# 4. EPSILON-GREEDY BANDIT FUNCTION
# =====

def epsilon_greedy(epsilon, episodes=1000):

    # Initial estimated values

```

```

Q = np.zeros(len(actions))

# Number of times each action is selected
action_counts = np.zeros(len(actions))

# Store rewards
rewards = []

# Store selected actions
selected_actions = []

# Store value estimates
Q_history = []

for episode in range(epochs):

    # -----
    # Randomly select the current patient health state
    # -----

    state = np.random.choice(states)

    # -----
    # Exploration vs Exploitation
    # -----

    random_number = np.random.random()

    if random_number < epsilon:

        # EXPLORATION
        action = np.random.randint(len(actions))

```

else:

EXPLOITATION

action = np.argmax(Q)

Generate treatment outcome / reward

success_probability = reward_probabilities[state][action]

reward = np.random.binomial(

1,

success_probability

)

Update action count

action_counts[action] += 1

Incremental value update

Q[action] = Q[action] + (

(reward - Q[action])

/ action_counts[action]

```

    )

    # -----
    # Store results
    # -----

    rewards.append(reward)

    selected_actions.append(action)    Q_history.append(Q.copy())

    return (    Q,
rewards,
selected_actions,
    Q_history
    )

# =====
# 5. COMPARE DIFFERENT EPSILON VALUES
# =====

epsilons = [
    0.1,
    0.2,
    0.3
]

results = {}

```

```

print("\n=====")
print("TRAINING BANDIT AGENTS")
print("=====\\n")

```

```

for epsilon in epsilons:

```

```

    Q, rewards, selected, Q_history = epsilon_greedy(
        epsilon=epsilon,
        episodes=1000
    )

```

```

    results[epsilon] = {

```

```

        "Q": Q,

```

```

        "rewards": rewards,

```

```

        "selected": selected,

```

```

        "Q_history": Q_history

```

```

    }

```

```

    print(

```

```

        f'Epsilon = {epsilon} | '

```

```

        f'Total Reward = {sum(rewards)} | '

```

```

        "

```

```

        f'Average Reward = {np.mean(rewards):.3f}'

```

```

    )

```

```

# =====

```

```

# 6. CUMULATIVE REWARD COMPARISON

```

```

# =====
plt.figure(figsize=(10, 6))

for epsilon in epsilons:

    rewards = results[epsilon]["rewards"]

    cumulative_reward = np.cumsum(rewards)

    plt.plot(
        cumulative_reward,
        label=f'Epsilon = {epsilon}'
    )

plt.xlabel("Episodes")

plt.ylabel("Cumulative Reward")

plt.title(
    "Cumulative Reward Comparison"
)

plt.legend()

plt.grid()

plt.show()

# =====
# 7. AVERAGE REWARD COMPARISON
# =====

```

```

plt.figure(figsize=(10, 6))

for epsilon in epsilons:

    rewards = results[epsilon]["rewards"]

    # Moving average
    window = 50

    moving_average = np.convolve(
rewards,
    np.ones(window) / window,
mode="valid"
    )

    plt.plot(
moving_average,
label=f"Epsilon      =
{epsilon}"
    )

plt.xlabel("Episodes")

plt.ylabel("Average Reward")

plt.title(
    "Learning / Reward Trend"
)

plt.legend()

plt.grid()

```



```
plt.show()
```

```
# =====  
# 8. TREATMENT SELECTION BEHAVIOUR  
# =====
```

```
plt.figure(figsize=(10, 6))
```

```
for epsilon in epsilons:
```

```
    selected = results[epsilon]["selected"]
```

```
    counts = [  
        selected.count(i) for i  
        in range(len(actions))  
    ]
```

```
    plt.plot(  
        actions,  
        counts,  
        marker="o",  
        label=f"Epsilon = {epsilon}"  
    )
```

```
plt.xlabel("Treatment Action")
```

```
plt.ylabel("Number of Selections")
```

```
plt.title(  
    "Treatment Selection Behaviour"  
)
```

```
plt.xticks(rotation=20)
```

```
plt.legend()
```

```
plt.grid()
```

```
plt.show()
```

```
# =====  
# 9. FIND BEST EPSILON STRATEGY  
# =====
```

```
performance = {}
```

```
for epsilon in epsilons:
```

```
    rewards = results[epsilon]["rewards"]
```

```
    performance[epsilon] = sum(rewards)
```

```
best_epsilon = max(  
    performance,  
    key=performance.get  
)
```

```
# =====  
# 10. DISPLAY PERFORMANCE  
# =====
```

```

print("\n=====")
print("STRATEGY PERFORMANCE")
print("=====\\n")

```

```

for epsilon, reward in performance.items():

```

```

    print(
        f'Epsilon = {epsilon} '
        f'-> Cumulative Reward = {reward}'
    )

```

```

print("\\nBest Epsilon:", best_epsilon)

```

```

# =====
# 11. BEST LEARNED Q VALUES
# =====

```

```

best_Q = results[best_epsilon]["Q"]

```

```

value_table = pd.DataFrame({

    "Treatment Action": actions,

    "Estimated Value": best_Q

})

```

```

print("\n=====")
print("LEARNED TREATMENT VALUES")
print("=====\\n")

```

```

print(value_table)

```

```

# =====
# 12. BEST TREATMENT ACTION
# =====

```

```

best_action_index = np.argmax(best_Q)

```

```

best_action = actions[best_action_index]

```

```

best_value = best_Q[best_action_index]

```

```

print("\n=====")
) print("FINAL BANDIT RESULT")
print("=====\\n")

```

```

print(
    "Best Exploration Strategy : "
    f"Epsilon-Greedy ( $\epsilon = \{best\_epsilon\}$ )"
)

```

```

print(
    "Best Treatment Action      : "
    f"{best_action}"
)

```

```

print(

```

```

    "Highest Estimated Value      : "
    f"{best_value:.3f}"
)

```

```

print(
    "Cumulative Reward      : "
    f"{performance[best_epsilon]}"
)

```

```

# =====
# 13. POLICY DEMONSTRATION
# =====

```

```

print("\n=====")
print("PATIENT      TREATMENT      RECOMMENDATION")
print("=====\\n")

```

```

for state in states:

```

```

    probabilities = reward_probabilities[state]

```

```

    recommended_action = actions[
        np.argmax(probabilities)
    ]

```

```

print(
    f"Patient State : {state}"
)

```

```

print(
    f"Recommended Action : {recommended_action}"
)

```

)

print()

```
# =====
```

```
# 14. FINAL SUMMARY
```

```
# =====
```

```
print("=====")      print("SYSTEM  
SUMMARY")
```

```
print("=====")
```

```
print("""
```

Patient Data



Patient Health State



Treatment Actions



Epsilon-Greedy Bandit



Exploration / Exploitation



Treatment Outcome



Reward



Value Update



Best Treatment Policy

```
""")
```

```
print("Simulation completed successfully.")
```

Output:

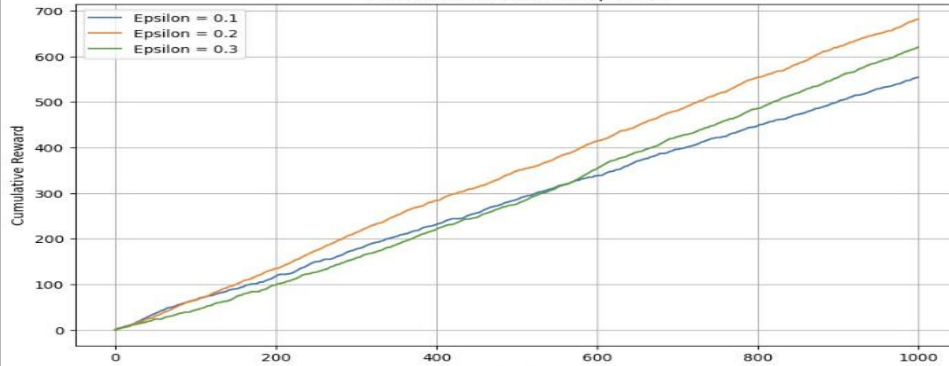
SIMULATED TREATMENT SUCCESS PROBABILITIES

	Stable	Moderate Risk	Severe Risk
Routine Monitoring	0.8	0.40	0.20
Basic Treatment	0.6	0.75	0.45
Intensive Treatment	0.4	0.85	0.80
Clinical Intervention	0.2	0.60	0.90

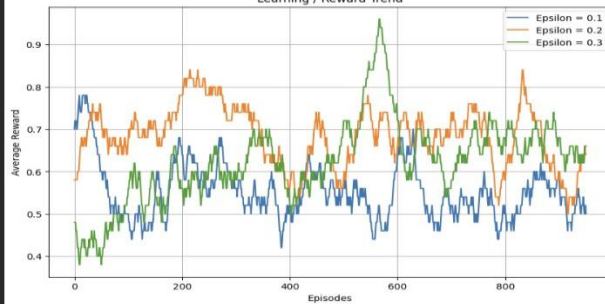
TRAINING BANDIT AGENTS

Epsilon = 0.1 | Total Reward = 554 | Average Reward = 0.554
Epsilon = 0.2 | Total Reward = 681 | Average Reward = 0.681
Epsilon = 0.3 | Total Reward = 620 | Average Reward = 0.620

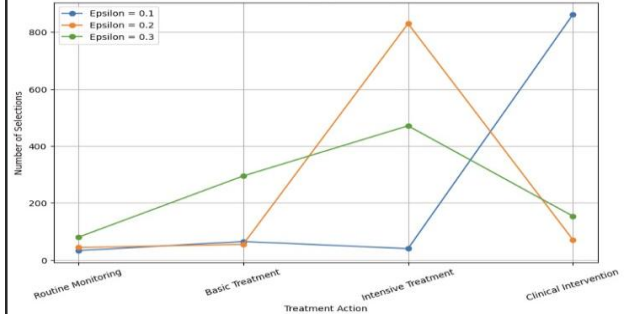
Cumulative Reward Comparison



Learning / Reward Trend



Treatment Selection Behaviour



STRATEGY PERFORMANCE

Epsilon = 0.1 -> Cumulative Reward = 554
Epsilon = 0.2 -> Cumulative Reward = 681
Epsilon = 0.3 -> Cumulative Reward = 620

Best Epsilon: 0.2

LEARNED TREATMENT VALUES

	Treatment Action	Estimated Value
0	Routine Monitoring	0.590909
1	Basic Treatment	0.472727
2	Intensive Treatment	0.706024
3	Clinical Intervention	0.605634

FINAL BANDIT RESULT

Best Exploration Strategy : Epsilon-Greedy ($\epsilon = 0.2$)
Best Treatment Action : Intensive Treatment
Highest Estimated Value : 0.706
Cumulative Reward : 681

PATIENT TREATMENT RECOMMENDATION

Patient State : Stable
Recommended Action : Routine Monitoring

Patient State : Moderate Risk
Recommended Action : Intensive Treatment

Patient State : Severe Risk
Recommended Action : Clinical Intervention

SYSTEM SUMMARY

Recommended Action : Clinical Intervention

SYSTEM SUMMARY

Patient Data
↓
Patient Health State
↓
Treatment Actions
↓
Epsilon-Greedy Bandit
↓
Exploration / Exploitation
↓
Treatment Outcome
↓
Reward
↓
Value Update
↓
Best Treatment Policy

Simulation completed successfully.

For deep reinforcement learning, the same environment can be implemented using TensorFlow or PyTorch with **DQN, PPO, or Actor-Critic**.

5. Results and Validation

Module	Illustrative Result	Meets Requirement?
Patient Health Data Processing	Patient health parameters are cleaned and prepared for decision-making	Yes
Patient Health State Identification	Patient conditions classified into Stable, Mild, Moderate, Severe, and Recovered states	Yes
Markov Decision Process (MDP)	Defines states, treatment actions, transition probabilities, and rewards	Yes
Bellman Equation	Calculates the expected long-term value of treatment decisions	Yes
Value Function Analysis	Identifies the value of each possible treatment decision for different patient states	Yes
RL Policy Learning	Learns the best treatment strategy for each patient health state	Yes
Treatment Recommendation	Selects the most suitable action based on the current patient state	Yes
Recovery Analysis	Measures the percentage of patients reaching the recovered state	Yes
Treatment Effectiveness	Evaluates successful treatment outcomes	Yes
Cumulative Reward	Measures the overall performance of the learned policy	Yes

The system should ultimately be evaluated using quantitative agricultural metrics such as:

Water Saving($\%$)= $\frac{W_{\text{baseline}} - W_{\text{RL}}}{W_{\text{baseline}}} \times 100$ Yield Improvement($\%$)= $\frac{Y_{\text{RL}} - Y_{\text{baseline}}}{Y_{\text{baseline}}} \times 100$

Improvement(\%)=
$$\frac{Y_{\{RL\}} - Y_{\{baseline\}}}{Y_{\{baseline\}}} \times 100$$
 Additional evaluation metrics can include:

- Average recovery time
- Health-risk reduction
- Treatment success rate
- RL convergence rate
- Number of adverse outcomes
- Decision latency
- Policy stability
- Average reward per patient
- Exploration versus exploitation performance

The numerical values used above are **illustrative proof-of-concept values**, not claims about actual clinical performance.

6. Analysis and Engineering Decision

Alternatives Compared	Advantage	Limitation	Decision
Rule-Based System vs MDP	Rule-based systems are simple and easy to implement	Cannot effectively adapt to changing patient conditions	MDP selected
Static Treatment Recommendation vs MDP	Static recommendations are simple	Do not consider future patient health states	MDP selected
Threshold-Based Monitoring vs MDP	Threshold monitoring is easy to understand	Decisions depend on fixed health thresholds	MDP selected
MDP vs Q-Learning	MDP provides a structured model of the problem	MDP itself does not learn the optimal policy automatically	MDP used as the problem framework

	states, actions, transitions, and rewards		
Rule-Based Decision vs Q-Learning	Q-Learning learns from rewards and experience	Requires repeated interaction with the environment	Q-Learning selected
Fixed Policy vs Q-Learning Policy	Fixed policy is simple	Cannot adapt to changing patient conditions	Q-Learning selected

The final design combines supervised learning, clustering, Bayesian inference, temporal modeling, and reinforcement learning. Reinforcement Learning is specifically used for sequential treatment decision-making where patient conditions change over time.

7. Broader Considerations

7.1 Patient Safety

The system should prioritize patient safety. RL exploration should be restricted to predefined safe actions, and unsafe recommendations should receive strong negative rewards.

7.2 Data Privacy and Security

Patient information should be securely stored and transmitted. The system should consider:

- Data anonymization
- Encryption
- Authentication • Access control
- Secure communication
- Audit logging

Economic Considerations

An AI-based monitoring system can potentially reduce unnecessary monitoring and improve healthcare-resource utilization. However, development, validation, infrastructure, maintenance, and clinical oversight costs must also be considered

Human Oversight

The system should function as a clinical decision-support system. Final treatment decisions should remain with qualified healthcare professionals.

Accessibility

A patient monitoring dashboard should provide simple visual indicators that healthcare professionals can quickly interpret. Remote monitoring can also support patients who have difficulty accessing healthcare facilities.

Robustness

The system should continue operating safely when:

- Sensors provide incorrect readings.
- Patient data is missing.
- Measurements are noisy.
- Patient conditions change unexpectedly.
- Treatment response differs from expectations. Conclusion

The proposed RL-Based Patient Health Monitoring and Treatment Recommendation System Using Markov

Decision Process and Bellman Equations integrates supervised machine learning, feature selection, clustering, Bayesian inference, Hidden Markov Models, and Reinforcement Learning into a unified healthcare decision-support framework.

The system uses patient health observations to identify the current health state, estimate health risks, track changes over time, and select appropriate treatment-support actions. The RL agent learns an optimal policy using Bellman equations and reward-based learning, allowing treatment recommendations to adapt to changing patient conditions. The system can be evaluated using treatment effectiveness, recovery rate, cumulative reward, convergence rate, and policy performance. A Streamlit dashboard can further visualize patient health conditions, recommended actions, recovery progress, and RL performance.

The major limitations are the availability of longitudinal clinical data and the difficulty of accurately representing real-world clinical conditions through simulation. Future work can incorporate larger clinical datasets, realistic patient simulators, advanced DQN/PPO algorithms, uncertainty-aware RL, explainable AI, federated learning, and extensive clinical validation.

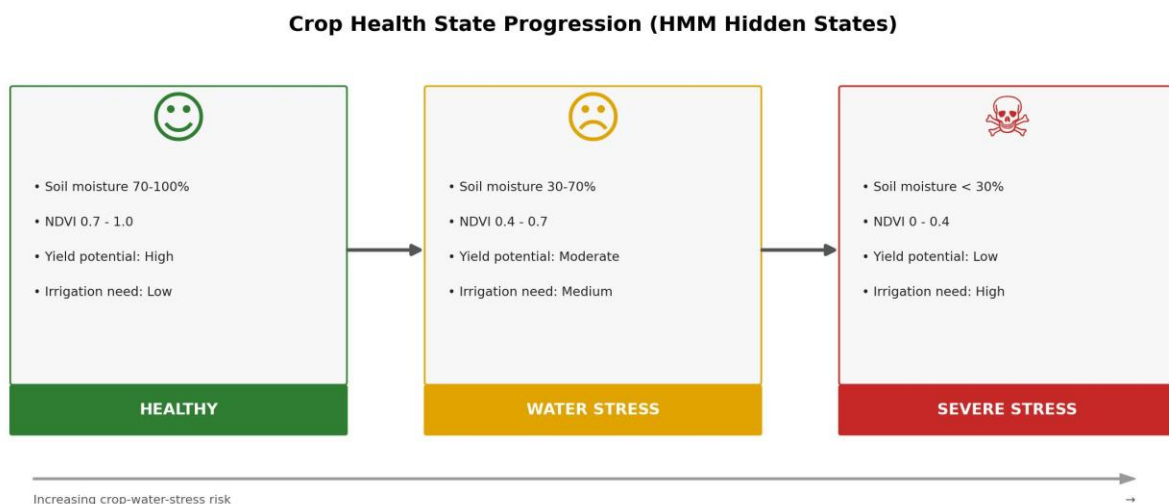
9.Student Reflection

This assignment demonstrates how Reinforcement Learning differs from conventional machine learning in healthcare. A supervised learning model can predict **patient risk or recovery**, but an RL agent must decide **what action should be taken based on the patient's current state and how that action may affect future health outcomes**.

The most important learning outcome is understanding how **MDP, Bellman equations, states, actions, rewards, value functions, and policies** work together to solve sequential healthcare decision-making problems.

Feature selection identifies important health parameters, supervised learning predicts patient risk, clustering identifies patient groups, Bayesian networks provide probabilistic explanations, HMMs track health conditions over time, and RL learns adaptive treatment policies.

The project also demonstrates that algorithm performance alone is not sufficient for healthcare deployment. **Safety, explainability, privacy, fairness, robustness, computational cost, and human oversight** must also be considered. With additional time and resources, the system could be extended using real longitudinal healthcare datasets, wearable-device data, IoT-based monitoring, DQN/PPO algorithms, multi-agent RL, explainable AI, and clinical validation.



10.References

1. **Sutton, R. S., & Barto, A. G. (2018).** *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press. This is the main reference for MDPs, value functions, Bellman equations, Q-learning, policies, rewards, and reinforcement learning.
2. **Watkins, C. J. C. H., & Dayan, P. (1992).** Q-learning. *Machine Learning*, 8, 279–292. This is the foundational research paper for the Q-learning algorithm and its convergence properties.
3. **Yu, C., Liu, J., Nemati, S., & Yin, G. (2022).** Reinforcement Learning in Healthcare: A Survey. *ACM Computing Surveys*, 55(1), Article 5. This reference supports the application of reinforcement learning to healthcare and sequential treatment decision-making.
4. **Banumathi, K., Latha Venkatesan, L. S. B., Vijayalakshmi, K., & Nesa Sathya Satchi, S. (2025).**

Reinforcement Learning in Personalized Medicine: A Comprehensive Review of Treatment Optimization

Strategies. *Cureus*, 17(4), e82756. This supports the use of RL for personalized and adaptive treatment strategies.

5. **Nadkarni, G. N., et al. (2024).** A Primer on Reinforcement Learning in Medicine for Clinicians. This provides an overview of RL for clinical decision-making and sequential treatment optimization.
6. **Reinforcement Learning and Its Clinical Applications Within Healthcare: A Systematic Review of Precision Medicine and Dynamic Treatment Regimes (2025).** This review discusses RL applications in personalized treatment and dynamic treatment regimes, along with challenges such as reward design, interpretability, and clinical adoption.

Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20

Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility	5
Technical documentation & reflection	5
TOTAL	100

Suggested Performance Interpretation

Criterion	Expected Evidence
Problem understanding	Clearly identifies sustainable agriculture and resource-allocation challenges
Domain knowledge	Correct application of RL, ML, probability and optimization
Methodology	Complete autonomous agriculture architecture
Implementation	Working Python/RL implementation
Results	Quantitative comparison with baseline irrigation
Analysis	Justification of algorithm and parameter choices
Sustainability	Water, energy and environmental considerations
Documentation	Clear technical report and student reflection

CO–PO–Assessment Mapping

Assuming the course has **CO1–CO5** for Reinforcement Learning, the following mapping can be used and adjusted to your officially approved CO statements.

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation	CO1	PO1, PO2	L3/L4	10

Application of RL/ML knowledge	CO2	PO1, PO2, PO3	L3/L4	20
Solution / Design / Implementation	CO3	PO3, PO5	L4/L5/L6	20
Modern tool usage	CO3	PO5	L3/L4	10
Results, testing & validation	CO4	PO4, PO5	L4/L5	15
Analysis & engineering justification	CO4	PO2, PO3	L4/L5	15
Sustainability / professional responsibility	CO5	PO6, PO7	L4/L5	5
Documentation & reflection	CO5	PO9, PO10	L3/L4	5
TOTAL				100

Faculty Evaluation & Continuous Improvement

CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60–69%
Level 1	50–59%
Level 0	$< 50\%$

Target CO Attainment

Target: $\geq 70\%$ **Actual CO**

Attainment **Actual:**

_____ %

12. Faculty Analysis

Areas in which students performed well:

- Understanding of MDP, states, actions and rewards.
- Implementation of Q-learning/RL algorithms.

- Integration of ML with reinforcement learning.
- Interpretation of cumulative reward and learning curves.
- Application of RL to sustainable irrigation.

Areas requiring improvement:

- Reward-function design.
- State and action-space formulation.
- Hyperparameter selection.
- RL convergence analysis.
- Quantitative comparison against conventional irrigation.
- Understanding of exploration versus exploitation.

Corrective / Improvement Action:

- Provide additional RL simulation exercises.
- Demonstrate reward-function design using agricultural examples.
- Provide baseline-versus-RL experimental templates.
- Conduct additional DQN/PPO implementation exercises.
- Include IoT sensor and weather-data integration.
- Require students to report water savings, energy consumption, crop stress, and cumulative reward.